

Quiz — 1.4.2 Data Structures

Name: _____ Date: _ **Mark:** _____ / 30

OCR H446 · Component 01 · 1.4.2 — show all working

Section A — Multiple choice (8 marks)

1. Which data structure operates on a **Last In, First Out** basis? A. Queue B. Stack C. Linked list D. Binary search tree Your answer: ☐
 2. Which statement best describes a **record**? A. An ordered collection of items of the same type accessed by index B. A collection of named fields that may hold different data types C. An immutable ordered sequence D. A set of nodes each pointing to the next Your answer: ☐
 3. Which is the key property of a **static** data structure? A. It grows and shrinks at run time B. Its size is fixed when it is declared C. It is always stored on the heap D. It cannot be accessed by index Your answer: ☐
 4. In a circular queue of size 8, the rear pointer is updated using which expression? A. `rear = rear + 8` B. `rear = (rear + 1) MOD 8` C. `rear = rear - 1` D. `rear = (rear - 1) MOD 8` Your answer: ☐
 5. Which traversal of a binary search tree visits the nodes in **ascending order**? A. Pre-order B. Post-order C. In-order D. Level-order Your answer: ☐
 6. What is a **collision** in a hash table? A. When the table becomes full B. When two different keys hash to the same index C. When a key cannot be hashed at all D. When the load factor reaches zero Your answer: ☐
 7. A major disadvantage of a **linked list** compared with an array is that it: A. Cannot grow at run time B. Must store items of different types C. Has no random access — it must be traversed from the head D. Uses no extra memory Your answer: ☐
 8. Which describes the correct ends used by a **queue**? A. Enqueue at the front, dequeue at the rear B. Enqueue and dequeue both at the top C. Enqueue at the rear, dequeue at the front D. Enqueue at the rear, dequeue at the rear Your answer: ☐
-

Section B — Calculations & short answer (17 marks)

9. Match each description to the correct data structure (Array, Tuple, Linked list, Hash table, Stack). [5]

#	Description	Structure
a	Ordered, immutable sequence such as <code>(x, y)</code>	—
b	LIFO structure with push/pop at the top	—
c	Fixed-size, same-type collection accessed by a numeric index	—
d	Nodes of data + a pointer to the next node	—
e	Maps a key to an index using a hash function for $\sim O(1)$ lookup	—
		(Linked list)

10. A **stack** is initially empty. The following operations are carried out in order. Trace the stack and give the values popped and the final contents. **[4]**

`push(5)` , `push(9)` , `push(2)` , `pop()` , `push(7)` , `pop()`

Working space (show the stack after each operation):

Values popped (in order): _ **Final stack (top → bottom):** _

11. A **queue** uses `enqueue` (rear) and `dequeue` (front). Starting empty, perform: **[3]**

`enqueue(A)` , `enqueue(B)` , `dequeue()` , `enqueue(C)` , `enqueue(D)` , `dequeue()`

Working space:

Values dequeued (in order): _ **Queue contents (front → rear):** _

12. The following numbers are inserted, in this order, into an empty **binary search tree**: **50, 30, 70, 20, 40, 60**. Draw the resulting tree. **[3]**

Working space:

13. State **two** differences between a **static** and a **dynamic** data structure. **[2]**

Working space:

Section C — Exam-style question (5 marks)

14. A web browser stores the list of pages the user has visited so the **Back** button can return to the previous page. Items are also held in a buffer that streams video, releasing frames in the order they arrived.

(a) Name the most appropriate data structure for the **Back button** history and justify your choice. **[2]**

(b) Name the most appropriate data structure for the **video buffer** and justify your choice. **[2]**

(c) State one reason a programmer might choose a **dynamic** implementation for either structure. **[1]**

Working space:

Answer key

Section A (8 marks — 1 each) 1. **B** — stack is LIFO. 2. **B** — record = named fields, possibly different types. 3. **B** — static size is fixed at declaration. 4. **B** — circular queue: $\text{rear} = (\text{rear} + 1) \text{ MOD size}$. 5. **C** — in-order (L, root, R) of a BST gives ascending order. 6. **B** — collision = two keys hash to the same index. 7. **C** — linked lists have no random access; must traverse from the head. 8. **C** — enqueue at rear, dequeue at front.

Section B

9. [5] (1 mark each) a = **Tuple** b = **Stack** c = **Array** d = **Linked list** e = **Hash table**

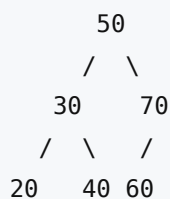
10. [4] Stack trace (top listed first): - push(5) → [5] - push(9) → [9, 5] - push(2) → [2, 9, 5] - pop() → returns **2** → [9, 5] - push(7) → [7, 9, 5] - pop() → returns **7** → [9, 5]

Values popped (in order): **2, then 7**. Final stack (top → bottom): **9, 5**. Mark: correct order of pushes onto top (1); first pop returns 2 (1); second pop returns 7 (1); correct final contents 9,5 (1).

11. [3] Queue trace (front listed first): - enqueue(A) → [A] - enqueue(B) → [A, B] - dequeue() → returns **A** → [B] - enqueue(C) → [B, C] - enqueue(D) → [B, C, D] - dequeue() → returns **B** → [C, D]

Values dequeued (in order): **A, then B**. Queue contents (front → rear): **C, D**. Mark: dequeue returns A then B in that order (1); FIFO applied at correct ends (1); final contents C, D (1).

12. [3] BST after inserting 50, 30, 70, 20, 40, 60:



- 50 is the root.
- $30 < 50 \rightarrow$ left of 50; $70 > 50 \rightarrow$ right of 50.
- $20 < 50, < 30 \rightarrow$ left of 30; $40 < 50, > 30 \rightarrow$ right of 30.
- $60 > 50, < 70 \rightarrow$ left of 70. Mark: 50 root with 30 left / 70 right (1); 20 left and 40 right of 30 (1); 60 as left child of 70 (1).

13. [2] Any two of: - Size: static is fixed at declaration; dynamic grows/shrinks at run time. (1) - Memory: static uses a contiguous block reserved up front; dynamic is allocated from the heap as needed. (1) - Trade-off: static gives fast/simple access but may waste or run out of memory; dynamic is flexible but adds pointer/management overhead. (1) (Max 2 marks.)

Section C

14. [5] (a) [2] A **stack** (1). The Back button needs the **most recently visited** page first — Last In, First Out — so pushing each new page and popping to go back returns pages in the correct reverse order (1).

(b) [2] A **queue** (1). A video buffer must release frames in the **order they arrived** — First In, First Out — so enqueueing incoming frames and dequeuing for playback preserves the correct sequence (1).

(c) [1] A dynamic implementation lets the structure **grow or shrink at run time**, so it adapts to an unknown/varying number of items without wasting memory or overflowing a fixed size (1).

Total: 8 + 17 + 5 = 30 marks.